MODI-SYNTH · Baseline Reader **B1** (SPAN-style FCN + row BiLSTM + CTC + count head)

Design: `docs/BASELINE_DESIGN.md`. This notebook trains **B1 on MoDeTrans** and evaluates on the test split.

Before you run anything (one time only)

1. On Google Drive, create the folder `MyDrive/modi/` and upload `colab_pack.zip` into it (from `FYP-modi-script-data/colab_pack.zip` on the laptop, 7.6 MB).
2. In Colab: **Runtime → Change runtime type → T4 GPU → Save**.
3. Run the cells **in order, top to bottom**. Each cell prints ✅ … when it worked.

If Colab disconnects: reconnect, run Cells 1–11 again, then Cell 13. Training **resumes** from the last saved epoch.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# Cell 1 — check that a GPU is attached
import torch
print('torch', torch.__version__)
assert torch.cuda.is_available(), 'No GPU! Runtime → Change runtime type → T4 GPU'
print('✅ GPU:', torch.cuda.get_device_name(0))
```

```
torch 2.11.0+cu130
✅ GPU: Tesla T4
```

```
# Cell 2 — Mount Google Drive and set all settings
from google.colab import drive
drive.mount('/content/drive')

import os, json

# Correct path to your folder in My Drive
DRIVE = '/content/drive/MyDrive/modi-script-research-drive/modi'
PACK_ZIP = f'{DRIVE}/colab_pack.zip'
DATA = '/content/data'

CFG = dict(
    run_name='B1_seed42', seed=42,
    img_h=512, img_w=1408,
    batch_size=8, accum=2,
    lr=3e-4, weight_decay=1e-2, warmup_steps=1000,
    max_epochs=400, patience=60, min_delta=0.1,
    use_lstm=True, use_count=True, use_aug=True, span_mask=True,
    count_weight=0.4, num_workers=2,
)

RUN_DIR = f"{DRIVE}/runs/{CFG['run_name']}"
os.makedirs(RUN_DIR, exist_ok=True)

assert os.path.exists(PACK_ZIP), (
    f'ZIP file not found at: {PACK_ZIP}. '
    'Please verify the folder and ZIP filename.'
)

print('✅ ZIP file found:', PACK_ZIP)
print('✅ Run folder:', RUN_DIR)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
✅ ZIP file found: /content/drive/MyDrive/modi-script-research-drive/modi/colab_pack.zip
✅ Run folder: /content/drive/MyDrive/modi-script-research-drive/modi/runs/B1_seed42
```

```
# Cell 3 — install the few libraries Colab does not have
!pip -q install datasets editdistance sacrebleu
print('✅ installed')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100.8/100.8 kB 9.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 129.6/129.6 kB 14.1 MB/s eta 0:00:00
✅ installed
```

```
# Cell 4 — unpack our metadata + folder-18 pages and verify every file's checksum
import hashlib, pathlib
!mkdir -p {DATA} && unzip -q -o "{PACK_ZIP}" -d {DATA}
man = json.load(open(f'{DATA}/MANIFEST.json'))
bad = [k for k, h in man['files'].items()
       if hashlib.sha256(pathlib.Path(DATA, k).read_bytes()).hexdigest() != h]
assert not bad, f'corrupt files: {bad}'
print(f"✅ pack OK: {len(man['files'])} files verified")
```

```
✅ pack OK: 94 files verified
```

```
# Cell 5 — load MoDeTrans straight from Hugging Face + our splits and cleaned text
from datasets import load_dataset
import pandas as pd

ds = load_dataset('historyHulk/MoDeTrans', split='train')        # 2,043 rows: filename, image, text
names = list(ds['filename'])
splits = json.load(open(f'{DATA}/metadata/splits.json', encoding='utf-8'))['modetrans']['assignment']
text = (pd.read_csv(f'{DATA}/metadata/modetrans_text.csv', keep_default_na=False)
          .set_index('image_id')['text_train'].to_dict())

assert len(names) == 2043 and set(names) == set(splits) == set(text), 'dataset / split mismatch'
SPLIT_IDS = {s: [n for n in names if splits[n] == s] for s in ['train', 'val', 'test']}
assert not (set(SPLIT_IDS['train']) & (set(SPLIT_IDS['val']) | set(SPLIT_IDS['test']))), 'leakage!'
print('✅ splits:', {k: len(v) for k, v in SPLIT_IDS.items()})
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
README.md: 100%  3.81k/3.81k [00:00<00:00, 315kB/s]
data/train-00000-of-00001.parquet: reconstructing file: 100%  161MB / 161MB, 15.4MB/s
data/train-00000-of-00001.parquet: downloading bytes:  159MB, 15.0MB/s
Generating train split: 100%  2043/2043 [00:00<00:00, 5148.50 examples/s]
✅ splits: {'train': 1430, 'val': 307, 'test': 306}
```

```
# Cell 6 — vocabulary: index 0 = CTC blank, then the 86 symbols, then <FRAC> and <unk>
import re
cs = json.load(open(f'{DATA}/metadata/charset.json', encoding='utf-8'))
VOCAB = ['<blank>'] + cs['symbols'] + ['<FRAC>', '<unk>']
STOI = {c: i for i, c in enumerate(VOCAB)}
NUM_CLASSES = len(VOCAB)
_TOK = re.compile(r'<FRAC>|.', re.S)

def encode(s):
    return [STOI.get(t, STOI['<unk>']) for t in _TOK.findall(s)]

def decode_ids(ids):
    return ''.join(VOCAB[i] for i in ids if i > 0)

bad = [n for n in names if decode_ids(encode(text[n])) != text[n]]
assert not bad, f'round-trip failed for {bad[:5]}'
n_unk = sum(t == STOI['<unk>'] for n in SPLIT_IDS['train'] for t in encode(text[n]))
assert n_unk == 0, 'train text contains unknown symbols'
MAX_LEN = max(len(encode(text[n])) for n in names)
print(f'✅ vocab {NUM_CLASSES} classes, longest label {MAX_LEN} tokens, all 2,043 labels round-trip')
```

```
✅ vocab 89 classes, longest label 452 tokens, all 2,043 labels round-trip
```

```
# Cell 7 — resize every patch once (height 512, aspect kept, white padding) and cache in RAM
import numpy as np, cv2
from tqdm.auto import tqdm
import matplotlib.pyplot as plt

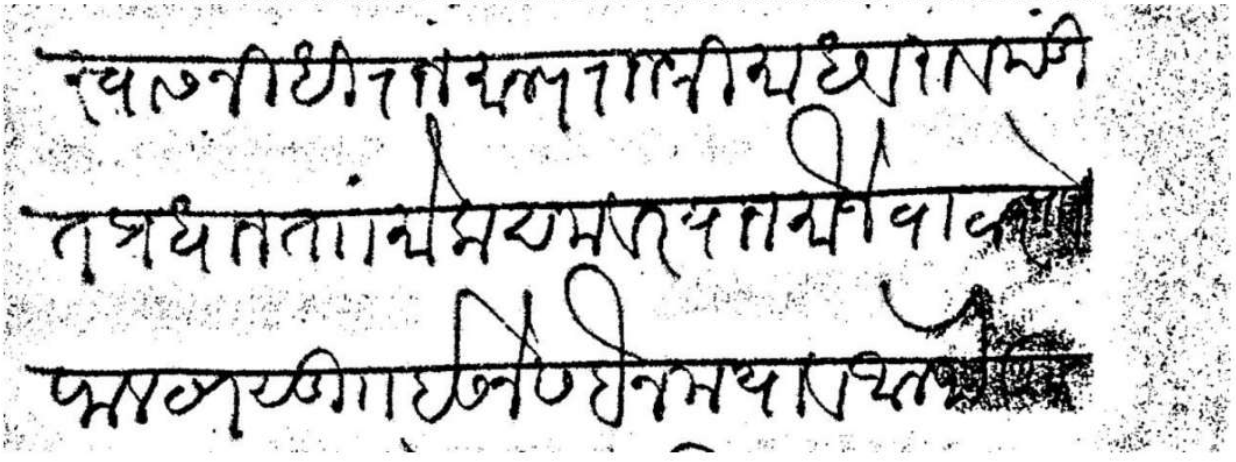
def prep(pil):
    g = np.array(pil.convert('L'))
    h, w = g.shape
    nw = min(round(w * CFG['img_h'] / h), CFG['img_w'])
    g = cv2.resize(g, (nw, CFG['img_h']), interpolation=cv2.INTER_LINEAR)
    canvas = np.full((CFG['img_h'], CFG['img_w']), 255, np.uint8)
    canvas[:, :nw] = g
    return canvas

CACHE = f"/content/cache_{CFG['img_h']}x{CFG['img_w']}.npy"
if os.path.exists(CACHE):
    IMGS = np.load(CACHE)
else:
    IMGS = np.stack([prep(r['image']) for r in tqdm(ds, desc='preprocessing')])
    np.save(CACHE, IMGS)
IDX = {n: i for i, n in enumerate(names)}
```

```
plt.figure(figsize=(14, 5)); plt.imshow(IMGS[IDX[SPLIT_IDS['train'][0]]], cmap='gray'); plt.axis('off')
plt.title(text[SPLIT_IDS['train'][0]][:90]); plt.show()
print('✅ cached', IMGS.shape, f'{IMGS.nbytes/1e9:.2f} GB')
```

```
preprocessing:  100%                                                2043/2043  [00:07<00:00,  210.96it/s]
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2360 (\N{DEVANAGARI LETTER SA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Matplotlib currently does not support
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2381 (\N{DEVANAGARI SIGN VIRAMA
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2357 (\N{DEVANAGARI LETTER VA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2366 (\N{DEVANAGARI VOWEL SIGN
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2344 (\N{DEVANAGARI LETTER NA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2367 (\N{DEVANAGARI VOWEL SIGN
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2343 (\N{DEVANAGARI LETTER DHA}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2368 (\N{DEVANAGARI VOWEL SIGN
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2352 (\N{DEVANAGARI LETTER RA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2332 (\N{DEVANAGARI LETTER JA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2350 (\N{DEVANAGARI LETTER MA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2351 (\N{DEVANAGARI LETTER YA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2358 (\N{DEVANAGARI LETTER SHA}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2346 (\N{DEVANAGARI LETTER PA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2306 (\N{DEVANAGARI SIGN ANUSVA
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2337 (\N{DEVANAGARI LETTER DDA}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2340 (\N{DEVANAGARI LETTER TA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2379 (\N{DEVANAGARI VOWEL SIGN
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2325 (\N{DEVANAGARI LETTER KA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2342 (\N{DEVANAGARI LETTER DA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2380 (\N{DEVANAGARI VOWEL SIGN
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2375 (\N{DEVANAGARI VOWEL SIGN
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2336 (\N{DEVANAGARI LETTER TTHA
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2347 (\N{DEVANAGARI LETTER PHA}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2354 (\N{DEVANAGARI LETTER LA})
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2335 (\N{DEVANAGARI LETTER TTA}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2339 (\N{DEVANAGARI LETTER NNA}
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2369 (\N{DEVANAGARI VOWEL SIGN
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.13/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2311 (\N{DEVANAGARI LETTER I})
  fig.canvas.print_figure(bytes_io, **kw)
```

□□□□□□□□□ □□□□□□□□ □□□□□□□ □□□□□□□ □□□□□ □□□□□□ □□□ □□□□□ □ □□□□ □□□□ □□□□□ □□□ □□□□ □□□ □



✅ cached (2043, 512, 1408) 1.47 GB

```
# Cell 8 — augmentation (train only) + Dataset + DataLoaders
import random
from torch.utils.data import Dataset, DataLoader

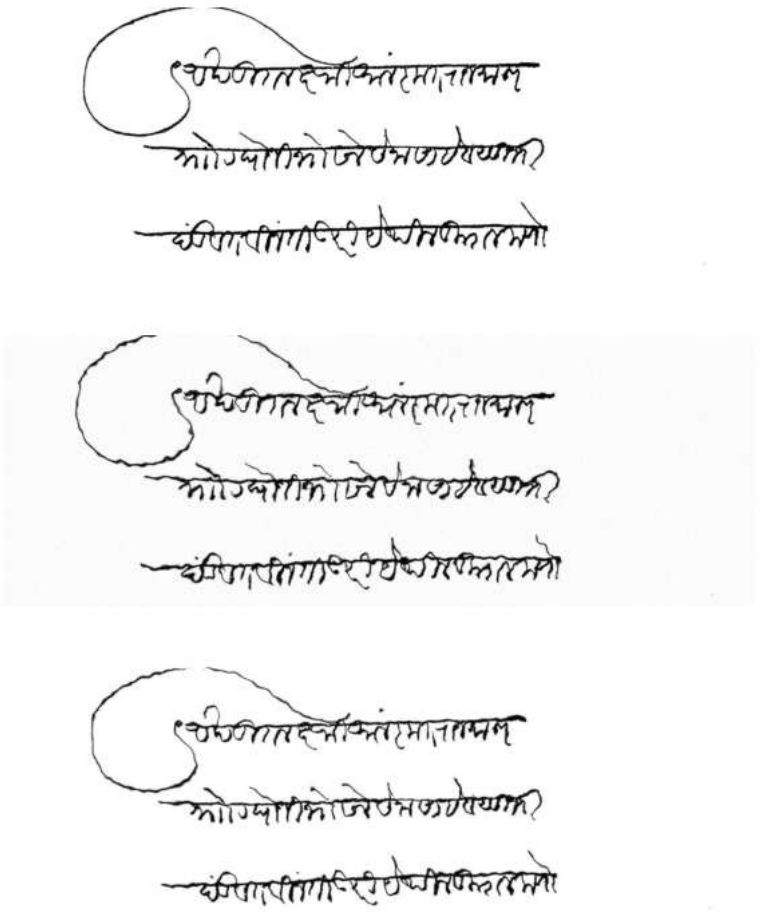
def augment(img, rng):
    h, w = img.shape
    out = img
    if rng.random() < 0.5:                                  # rotation ±2°, shear, scale, shift
        M = cv2.getRotationMatrix2D((w / 2, h / 2), rng.uniform(-2, 2), rng.uniform(0.92, 1.05))
        sh = rng.uniform(-0.15, 0.15)
        M[0, 1] += sh; M[0, 2] -= sh * h / 2
        M[0, 2] += rng.uniform(-20, 20); M[1, 2] += rng.uniform(-10, 10)
        out = cv2.warpAffine(out, M, (w, h), flags=cv2.INTER_LINEAR, borderValue=255)
    if rng.random() < 0.5:                                  # mild perspective
        d = 0.03
        src = np.float32([[0, 0], [w, 0], [w, h], [0, h]])
        dst = src + np.float32(rng.uniform(-d, d, (4, 2)) * [w, h])
        out = cv2.warpPerspective(out, cv2.getPerspectiveTransform(src, dst), (w, h), borderValue=255)
    if rng.random() < 0.5:                                  # stroke width: erode = thicker ink, dilate = thinner
        k = np.ones((2, 2), np.uint8)
        out = cv2.erode(out, k) if rng.random() < 0.5 else cv2.dilate(out, k)
    if rng.random() < 0.3:                                  # elastic
        a, s = rng.uniform(30, 60), rng.uniform(5, 7)
        dx = cv2.GaussianBlur((rng.random((h, w)) * 2 - 1).astype(np.float32), (0, 0), s) * a
        dy = cv2.GaussianBlur((rng.random((h, w)) * 2 - 1).astype(np.float32), (0, 0), s) * a
        gx, gy = np.meshgrid(np.arange(w, dtype=np.float32), np.arange(h, dtype=np.float32))
        out = cv2.remap(out, gx + dx, gy + dy, cv2.INTER_LINEAR, borderValue=255)
    if rng.random() < 0.5:                                  # gaussian noise
        out = np.clip(out + rng.normal(0, rng.uniform(0, 12), out.shape), 0, 255).astype(np.uint8)
    return out

class MoDeTransDS(Dataset):
    def __init__(self, ids, train, aug):
        self.ids, self.train, self.aug = ids, train, aug
    def __len__(self):
        return len(self.ids)
    def __getitem__(self, i):
        n = self.ids[i]
        img = IMGS[IDX[n]]
        if self.train and self.aug:
            img = augment(img, np.random.default_rng(torch.randint(0, 2**31 - 1, (1,)).item()))
        x = torch.from_numpy(np.ascontiguousarray(img)).float().div(255).sub(0.5).div(0.5).unsqueeze(0)
        return x, torch.tensor(encode(text[n]), dtype=torch.long), n

def collate(batch):
    xs, ys, ns = zip(*batch)
    return torch.stack(xs), torch.cat(ys), torch.tensor([len(y) for y in ys]), list(ns)

def make_loader(ids, train, cfg=None):
    cfg = cfg or CFG
    bs, nw = cfg['batch_size'], cfg['num_workers']
    return DataLoader(MoDeTransDS(ids, train, cfg['use_aug']), batch_size=bs, shuffle=train,
                      drop_last=train and len(ids) > bs, num_workers=nw, collate_fn=collate,
                      pin_memory=torch.cuda.is_available(), persistent_workers=nw > 0)

rng = np.random.default_rng(0)
ex = IMGS[IDX[SPLIT_IDS['train'][1]]]
fig, ax = plt.subplots(3, 1, figsize=(14, 9))
for a, im in zip(ax, [ex, augment(ex, rng), augment(ex, rng)]):
    a.imshow(im, cmap='gray'); a.axis('off')
plt.show()
print('✅ data pipeline ready (top = original, below = two random augmentations)')
```



✅ data pipeline ready (top = original, below = two random augmentations)

```
# Cell 9 — the model: SPAN-style FCN encoder -> span masking -> per-row BiLSTM -> CTC head (+ count head)
import torch.nn as nn, torch.nn.functional as F

class ConvBlock(nn.Module):
    def __init__(self, cin, cout, stride, drop=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(cin, cout, 3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(cout, cout, 3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(cout, cout, 3, stride=stride, padding=1), nn.ReLU(inplace=True),
            nn.BatchNorm2d(cout), nn.Dropout2d(drop))
    def forward(self, x):
        return self.net(x)

class DSCBlock(nn.Module):                      # depthwise-separable residual block
    def __init__(self, c, drop=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(c, c, 3, padding=1, groups=c), nn.Conv2d(c, c, 1), nn.ReLU(inplace=True),
            nn.Conv2d(c, c, 3, padding=1, groups=c), nn.Conv2d(c, c, 1), nn.ReLU(inplace=True),
            nn.BatchNorm2d(c), nn.Dropout2d(drop))
    def forward(self, x):
        return x + self.net(x)

class Encoder(nn.Module):                       # downsamples height x32, width x8
    def __init__(self, d=256):
        super().__init__()
        self.net = nn.Sequential(
            ConvBlock(1, 32, (1, 1)), ConvBlock(32, 64, (2, 2)), ConvBlock(64, 128, (2, 2)),
            ConvBlock(128, d, (2, 2)), ConvBlock(d, d, (2, 1)), ConvBlock(d, d, (2, 1)),
            DSCBlock(d), DSCBlock(d), DSCBlock(d), DSCBlock(d))
    def forward(self, x):
        return self.net(x)

def span_mask(f, ratio=0.1, max_span=8):
    B, D, H, W = f.shape
    m = torch.ones(B, 1, 1, W, device=f.device, dtype=f.dtype)
    n = max(1, int(W * ratio / ((max_span + 1) / 2)))
    for b in range(B):
        for _ in range(n):
            L = random.randint(1, max_span); s = random.randint(0, W - L)
            m[b, :, :, s:s + L] = 0
```

```
    return f * m

class Reader(nn.Module):
    def __init__(self, n_classes, use_lstm=True, use_count=True, use_span_mask=True, d=256):
        super().__init__()
        self.enc = Encoder(d)
        self.use_span_mask = use_span_mask
        self.lstm = nn.LSTM(d, d // 2, num_layers=2, bidirectional=True, batch_first=True, dropout=0.1) if use_lstm else No
        self.head = nn.Conv2d(d, n_classes, 1)
        self.count = nn.Sequential(nn.Linear(d, 128), nn.ReLU(inplace=True), nn.Linear(128, 1)) if use_count else None
    def forward(self, x):
        f = self.enc(x)                                   # B, D, H/32, W/8
        if self.training and self.use_span_mask:
            f = span_mask(f)
        if self.lstm is not None:                         # BiLSTM along each feature row
            B, D, H, W = f.shape
            r, _ = self.lstm(f.permute(0, 2, 3, 1).reshape(B * H, W, D))
            f = r.reshape(B, H, W, D).permute(0, 3, 1, 2)
        logits = self.head(f)                             # B, C, H, W
        B, C, H, W = logits.shape
        seq = logits.permute(0, 2, 3, 1).reshape(B, H * W, C) # rows concatenated top->bottom
        cnt = self.count(f.mean((2, 3))).squeeze(1) if self.count is not None else None
        return seq, cnt

DEV = 'cuda' if torch.cuda.is_available() else 'cpu'
_m = Reader(NUM_CLASSES, CFG['use_lstm'], CFG['use_count'], CFG['span_mask']).to(DEV).eval()
with torch.no_grad():
    _seq, _ = _m(torch.zeros(1, 1, CFG['img_h'], CFG['img_w'], device=DEV))
N_FRAMES = _seq.shape[1]
print(f'params: {sum(p.numel() for p in _m.parameters())/1e6:.1f} M, CTC frames per patch: {N_FRAMES}')

# CTC needs frames >= label length + number of adjacent repeated symbols
need = max(len(encode(text[n])) + sum(a == b for a, b in zip(encode(text[n]), encode(text[n])[1:])) for n in names)
assert N_FRAMES >= need, f'not enough CTC frames ({N_FRAMES} < {need})'
del _m, _seq
print(f'✅ model OK, frame budget {N_FRAMES} >= worst-case need {need}')
```

```
params: 6.9 M, CTC frames per patch: 2816
✅ model OK, frame budget 2816 >= worst-case need 452
```

```
# Cell 10 — decoding and metrics (CER primary, WER, chrF++, BLEU, median CER, catastrophic rate, bootstrap CI)
import editdistance, sacrebleu

_FR = '⍰'                                  # <FRAC> counts as ONE character
def _u(s): return s.replace('<FRAC>', _FR)

def greedy_decode(seq):                          # seq: B, T, C
    ids = seq.argmax(-1).cpu().numpy()
    keep = np.ones_like(ids, dtype=bool)
    keep[:, 1:] = ids[:, 1:] != ids[:, :-1]
    keep &= ids != 0
    return [decode_ids(row[k]) for row, k in zip(ids, keep)]

def compute_metrics(refs, hyps, n_boot=0, seed=0):
    e = np.array([editdistance.eval(_u(r), _u(h)) for r, h in zip(refs, hyps)], float)
    L = np.array([max(1, len(_u(r))) for r in refs], float)
    we = np.array([editdistance.eval(r.split(), h.split()) for r, h in zip(refs, hyps)], float)
    wl = np.array([max(1, len(r.split())) for r in refs], float)
    per = e / L
    out = dict(CER=100 * e.sum() / L.sum(), WER=100 * we.sum() / wl.sum(),
               median_CER=100 * float(np.median(per)), catastrophic_pct=100 * float((per > 0.5).mean()),
               chrF_pp=sacrebleu.corpus_chrf(hyps, [refs], word_order=2).score,
               BLEU=sacrebleu.corpus_bleu(hyps, [refs]).score, n=len(refs))
    if n_boot:
        rng = np.random.default_rng(seed)
        idx = rng.integers(0, len(refs), (n_boot, len(refs)))
        b = 100 * e[idx].sum(1) / L[idx].sum(1)
        out['CER_CI95'] = (float(np.percentile(b, 2.5)), float(np.percentile(b, 97.5)))
    return out

print('✅ metrics self-test:', compute_metrics(['राम आला'], ['राम आल']))
```

```
': np.float64(14.285714285714286), 'WER': np.float64(50.0), 'median_CER': 14.285714285714285, 'catastrophic_pct': 0.0, 'chrF
```

```
# Cell 11 — training / evaluation / checkpoint-resume logic
import math, time, csv, copy

def set_seed(s):
    random.seed(s); np.random.seed(s); torch.manual_seed(s); torch.cuda.manual_seed_all(s)

def make_scaler(enabled):
```

```
        try:
            return torch.amp.GradScaler('cuda', enabled=enabled)
        except (AttributeError, TypeError):
            return torch.cuda.amp.GradScaler(enabled=enabled)

    CTC = nn.CTCLoss(blank=0, zero_infinity=True)

    def loss_fn(seq, cnt, y, yl):
        logp = seq.float().log_softmax(-1).permute(1, 0, 2)              # T, B, C (CTC in fp32)
        il = torch.full((logp.size(1),), logp.size(0), dtype=torch.long)
        loss = CTC(logp, y, il, yl)
        if cnt is not None:
            loss = loss + CFG_RUN['count_weight'] * F.l1_loss(cnt.float(), yl.float().to(cnt.device) / 100)
        return loss

    @torch.no_grad()
    def evaluate(model, loader):
        model.eval(); refs, hyps, ns = [], [], []
        for x, y, yl, n in loader:
            with torch.autocast(device_type=DEV, dtype=torch.float16, enabled=(DEV == 'cuda')):
                seq, _ = model(x.to(DEV, non_blocking=True))
            hyps += greedy_decode(seq); refs += [text[k] for k in n]; ns += n
        return refs, hyps, ns

    def fit(cfg, train_ids, val_ids, run_dir, verbose_every=1, stop_at_cer=None):
        global CFG_RUN
        CFG_RUN = cfg
        os.makedirs(run_dir, exist_ok=True)
        set_seed(cfg['seed'])
        model = Reader(NUM_CLASSES, cfg['use_lstm'], cfg['use_count'], cfg['span_mask']).to(DEV)
        tl = make_loader(train_ids, True, cfg); vl = make_loader(val_ids, False, cfg)
        opt = torch.optim.AdamW(model.parameters(), lr=cfg['lr'], weight_decay=cfg['weight_decay'])
        total = max(1, cfg['max_epochs'] * len(tl) // cfg['accum'])
        warm = min(cfg['warmup_steps'], total // 10 + 1)
        sched = torch.optim.lr_scheduler.LambdaLR(opt, lambda s: (s + 1) / warm if s < warm else
            max(1e-6 / cfg['lr'], 0.5 * (1 + math.cos(math.pi * (s - warm) / max(1, total - warm)))))
        scaler = make_scaler(DEV == 'cuda')
        state = dict(epoch=0, best_cer=1e9, bad_epochs=0)
        last = f'{run_dir}/last.pt'
        if os.path.exists(last):                                    # ---- resume ----
            ck = torch.load(last, map_location=DEV, weights_only=False)
            model.load_state_dict(ck['model']); opt.load_state_dict(ck['opt'])
            sched.load_state_dict(ck['sched']); scaler.load_state_dict(ck['scaler']); state = ck['state']
            print(f"↻ resumed from epoch {state['epoch']} (best val CER {state['best_cer']:.2f})")
        json.dump(cfg, open(f'{run_dir}/config.json', 'w'), indent=2)
        log_path = f'{run_dir}/log.csv'
        if not os.path.exists(log_path):
            csv.writer(open(log_path, 'w', newline='')).writerow(['epoch', 'train_loss', 'val_CER', 'val_WER', 'lr', 'sec'])

        while state['epoch'] < cfg['max_epochs'] and state['bad_epochs'] < cfg['patience']:
            t0 = time.time(); model.train(); tot, nb = 0.0, 0
            opt.zero_grad(set_to_none=True)
            for step, (x, y, yl, _) in enumerate(tl):
                with torch.autocast(device_type=DEV, dtype=torch.float16, enabled=(DEV == 'cuda')):
                    seq, cnt = model(x.to(DEV, non_blocking=True))
                loss = loss_fn(seq, cnt, y, yl)
                scaler.scale(loss / cfg['accum']).backward()
                if (step + 1) % cfg['accum'] == 0:
                    scaler.unscale_(opt); torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
                    scaler.step(opt); scaler.update(); opt.zero_grad(set_to_none=True); sched.step()
                tot += loss.item(); nb += 1
            refs, hyps, _ = evaluate(model, vl)
            m = compute_metrics(refs, hyps)
            state['epoch'] += 1
            improved = m['CER'] < state['best_cer'] - cfg['min_delta']
            if improved:
                state['best_cer'] = m['CER']; state['bad_epochs'] = 0
                torch.save({'model': model.state_dict(), 'cfg': cfg, 'epoch': state['epoch'], 'val': m}, f'{run_dir}/best.pt'
            else:
                state['bad_epochs'] += 1
            torch.save({'model': model.state_dict(), 'opt': opt.state_dict(), 'sched': sched.state_dict(),
                        'scaler': scaler.state_dict(), 'state': state}, last)
            lr = opt.param_groups[0]['lr']; sec = time.time() - t0
            csv.writer(open(log_path, 'a', newline='')).writerow([state['epoch'], round(tot / nb, 4), round(m['CER'], 2), rou
            if state['epoch'] % verbose_every == 0 or improved:
                print(f"ep {state['epoch']:3d} | loss {tot/nb:7.3f} | val CER {m['CER']:6.2f} WER {m['WER']:6.2f} | "
                      f"best {state['best_cer']:6.2f} | lr {lr:.1e} | {sec:.0f}s {'★' if improved else ''}")
                print('   GT  :', refs[0][:80]); print('   PRED:', hyps[0][:80])
            if stop_at_cer is not None and m['CER'] <= stop_at_cer:
                print(f'reached CER {m["CER"]:.2f} <= {stop_at_cer}, stopping'); break
        print(f"✅ done: epochs {state['epoch']}, best val CER {state['best_cer']:.2f} → {run_dir}/best.pt")
```

```
training functions ready
```

```
# Cell 12 — SANITY TEST: the model must memorise 20 training patches (CER near 0).
# If this does not reach CER < 5 within ~300 epochs, STOP and debug before full training.
sanity_cfg = dict(CFG, run_name='overfit20', use_aug=False, span_mask=False, batch_size=4, accum=1,
                  lr=1e-3, warmup_steps=50, max_epochs=400, patience=400, num_workers=0)
ids20 = SPLIT_IDS['train'][:20]
_ = fit(sanity_cfg, ids20, ids20, '/content/overfit20', verbose_every=25, stop_at_cer=5.0)
```

```
   GT  : स्वासनिधी राजमान्य राजश्री माधवराव पंडित प्रधान तां मोकदम व रयान मौजे वाठार पाा
   PRED: आपआै
ep   8 | loss   5.802 | val CER  96.67 WER 100.00 | best  96.67 | lr 8.2e-04 | 3s ★
   GT  : स्वासनिधी राजमान्य राजश्री माधवराव पंडित प्रधान तां मोकदम व रयान मौजे वाठार पाा
   PRED:
ep   9 | loss   5.012 | val CER  91.16 WER 100.00 | best  91.16 | lr 9.2e-04 | 3s ★
   GT  : स्वासनिधी राजमान्य राजश्री माधवराव पंडित प्रधान तां मोकदम व रयान मौजे वाठार पाा
   PRED:
ep  25 | loss   3.482 | val CER 100.00 WER 100.00 | best  91.16 | lr 1.0e-03 | 3s
   GT  : स्वासनिधी राजमान्य राजश्री माधवराव पंडित प्रधान तां मोकदम व रयान मौजे वाठार पाा
   PRED:
ep  50 | loss   3.476 | val CER  99.96 WER 100.00 | best  91.16 | lr 9.7e-04 | 4s
   GT  : स्वासनिधी राजमान्य राजश्री माधवराव पंडित प्रधान तां मोकदम व रयान मौजे वाठार पाा
   PRED:
ep  75 | loss   3.447 | val CER 100.00 WER 100.00 | best  91.16 | lr 9.3e-04 | 4s
   GT  : स्वासनिधी राजमान्य राजश्री माधवराव पंडित प्रधान तां मोकदम व रयान मौजे वाठार पाा
```